

Autonomous Articulated Vehicle Control via Deep Reinforcement Learning and High-Dimensional Perception

Benjamin Schnapp

March 30, 2026

Contents

1	Introduction	6
1.1	Motivation	6
1.2	Problem Definition	6
1.3	Thesis Contributions	7
1.4	Institutional Context	7
1.5	Thesis Organisation	8
2	Literature Review	9
2.1	Articulated Vehicle Control	9
2.1.1	Classical Stabilisation Techniques	9
2.1.2	Kinematic Models	9
2.2	Reinforcement Learning in Autonomous Driving	10
2.2.1	Policy Gradient Methods	10
2.2.2	Transformer-Based Methods	10
2.3	Perception and Representation Learning	10
2.3.1	Convolutional Neural Networks for Driving	10
2.3.2	Autoencoders for State Compression	10
2.4	Safety in Autonomous Systems	11
2.5	Benchmarking Against Recent Waterloo Theses	11
3	Mathematical Modeling and Problem Formulation	13
3.1	Tractor Dynamics	13
3.2	Trailer Articulation and Reverse Instability	13
3.3	Challenges of Reverse Maneuvering	14
3.4	MDP Formulation	14
4	Perception and Observation Space Design	15
4.1	Observation Modalities	15
4.2	Bird's-Eye View Observation	15
4.3	CNN Encoder	16
4.4	Autoencoder Latent Space	16

4.5	UNet Encoder Variant	17
4.6	Lidar Observation	17
4.7	Observation Space Summary	17
5	Reinforcement Learning Framework	19
5.1	Twin Delayed Deep Deterministic Policy Gradient (TD3)	19
5.1.1	Reward Function	19
5.1.2	Multiplicative vs. Additive Reward Structures	20
5.2	Decision Transformer	20
5.2.1	Dataset Requirements	20
5.3	Hyperparameter Optimisation	21
5.3.1	Representation Pretraining	21
5.3.2	Classical Controller Tuning	21
6	Simulation Environment	23
6.1	Custom Pygame/Gymnasium Environment	23
6.2	Environment Tasks	24
6.2.1	Line Following	24
6.2.2	Obstacle Avoidance	24
6.3	Vehicle Parameters	25
6.4	Action Space	25
6.5	Testing Protocols	26
6.5.1	Lane Following Performance	27
6.5.2	Obstacle Avoidance	27
6.5.3	Comparison with Prior Work	27
7	Results and Discussion	29
7.1	Path Following Performance	29
7.2	Reverse Maneuvering Success	30
7.3	Perception Robustness	31
7.4	Benchmark Comparison	31
8	Conclusion	33
8.1	Summary of Findings	33
8.2	Limitations	33
8.3	Future Work	34
A	Simulation Software Architecture	36
B	Full Hyperparameter Tables	37

C Additional Plots	38
D Reward Function Derivation	39

List of Figures

List of Tables

1.1	MASc thesis requirements summary	8
2.1	Selected recent MASc theses, University of Waterloo Mechatronics Engineering	11
4.1	Observation space components	17
6.1	Vehicle parameters used in simulation	25
6.2	Lane following experiment definitions	27
7.1	Planned run matrix for the experimental campaign	30
7.2	Controller performance comparison	31
7.3	Recommended experiment matrix for the thesis	32
B.1	TD3 Hyperparameters	37

Chapter 1

Introduction

1.1 Motivation

The increasing demand for autonomous logistics and the growth of large-scale goods transportation have positioned articulated tractor-trailer vehicles as a critical target for autonomous control research. The transition from rigid-body autonomous navigation to the control of articulated tractor-trailer systems represents a significant escalation in mechanical complexity, perception requirements, and control instability, particularly during reverse maneuvering.

Unlike single-body vehicles, tractor-trailer systems introduce non-holonomic constraints at the hitch point that fundamentally alter the stability characteristics of the vehicle. In reverse motion, the articulation angle γ between the tractor and trailer exhibits open-loop instability: absent precise closed-loop control, any perturbation grows unbounded, culminating in the well-known jackknife condition from which no steering correction can recover the trailer’s alignment. Classical approaches such as Nonlinear Model Predictive Control (NMPC) can manage this instability by optimising over a prediction horizon, but their computational demands and reliance on accurate system models limit their applicability in unstructured or sensor-rich environments.

1.2 Problem Definition

The core challenge addressed in this thesis is twofold. First, the kinematic and dynamic complexity of tractor-trailer systems makes it difficult to design reward functions and control policies that simultaneously optimise path tracking performance and hitch stability. Second, real-world autonomous deployment requires perception of lane boundaries and obstacles directly from high-dimensional sensor data, rather than from pre-computed symbolic state representations.

Prior work in the ME 780 project evaluated Deep Deterministic Policy Gradient

(DDPG) and Decision Transformer (DT) models for path following in single-vehicle systems. While DDPG achieved cross-track errors competitive with PID control, both learning-based methods showed significant degradation under novel conditions, and neither was tested on articulated dynamics or reverse motion.

1.3 Thesis Contributions

This thesis extends the ME 780 and ECE 682 foundational work by:

- developing a full kinematic and dynamic model of a tractor-trailer system including hitch instability analysis, building on the bicycle model and Pacejka tire dynamics established in prior work;
- designing a vision-based RL observation space using CNN encoders and autoencoders operating on bird’s-eye-view (BEV) images rendered from a custom Pygame simulation environment;
- engineering a reward function that balances cross-track error, heading error, hitch articulation stability, and collision avoidance;
- implementing and benchmarking Twin Delayed DDPG (TD3) agents in lane-following and obstacle-avoidance tasks within a custom Gymnasium-based simulation environment;
- comparing TD3 performance with classical control baselines (PID, MPC) and the prior ME 780 DDPG results across path-following metrics.

The research aligns with the objectives of the Mechatronics Vehicle Systems (MVS) Lab and the WATonoBus project at the University of Waterloo.

1.4 Institutional Context

This work is submitted in partial fulfilment of the requirements for a Master of Applied Science (MASc) degree in the Department of Mechanical and Mechatronics Engineering at the University of Waterloo. The degree requires six terms of full-time registration, completion of 3–5 graduate-level courses, appointment of two additional faculty readers, and a mandatory 15-day public display period. The target completion date is early July, with a seminar scheduled for late July.

Table 1.1: MASc thesis requirements summary

Element	Requirement
Research Scope	Substantial problem extending beyond existing laboratory results
Literature Review	Comprehensive assessment of classical control and modern AI approaches
Validation	Simulation-based empirical evidence across multiple environment configurations
Length/Format	Typically 80–120 pages; GSPA formatting and UWSpace submission
Committee	Supervisor plus two faculty readers; formal display period

1.5 Thesis Organisation

The remainder of this thesis is structured as follows. Chapter 2 reviews classical and learning-based control methods for articulated vehicles, alongside relevant perception and safety literature. Chapter 3 derives the state-space equations for the tractor-trailer system and analyses the jackknife instability. Chapter 4 presents the CNN and autoencoder-based perception stack. Chapter 5 defines the MDP formulation and details the DDPG and Decision Transformer implementations. Chapter 6 describes the custom Pygame/-Gymnasium simulation environment and experimental setup. Chapter 7 presents and discusses experimental results. Chapter 8 summarises the findings and outlines future work.

Chapter 2

Literature Review

2.1 Articulated Vehicle Control

2.1.1 Classical Stabilisation Techniques

Classical articulated vehicle control methods address the hitch instability through structured model-based approaches:

- **PID control:** Simple to implement and computationally negligible, but incapable of anticipating the jackknife condition. Gains must be re-tuned for different payloads and path curvatures.
- **Model Predictive Control (MPC):** Optimises steering over a receding horizon subject to state and input constraints. Linear MPC can enforce hard bounds on the articulation angle γ .
- **Nonlinear MPC (NMPC):** Captures the full nonlinear kinematics including reverse instability. Optimises steering over a receding horizon subject to state and input constraints, enabling hard bounds on the articulation angle γ , but with higher computational cost than linear methods.
- **Active hitch stabilisation:** Applies an additional actuated degree of freedom at the hitch to damp articulation oscillations. Research at Waterloo into trailer sway control and active hitches demonstrates that lateral articulation at the hitch can mitigate oscillations, but for a standard autonomous tractor-trailer this must be managed purely through the tractor's steering angle δ .

2.1.2 Kinematic Models

The bicycle model provides a baseline for rigid-body longitudinal and lateral dynamics. For an articulated system, the state vector must be expanded to include the trailer heading and articulation angle, as detailed in Chapter 3.

2.2 Reinforcement Learning in Autonomous Driving

2.2.1 Policy Gradient Methods

Deep Deterministic Policy Gradient (DDPG) is an off-policy actor-critic algorithm designed for continuous action spaces. Prior work in ME 780 established that DDPG achieves a cross-track error of approximately 0.505 m for rigid-body path following, outperforming a PID baseline of 0.784 m on the same task.

Proximal Policy Optimisation (PPO) offers improved stability over DDPG through clipped probability ratios, but its on-policy sample efficiency is lower, which is a significant consideration when simulation rollouts are expensive.

2.2.2 Transformer-Based Methods

Decision Transformers (DT) reformulate RL as sequence modelling: an action is predicted from a context window of past states, actions, and returns-to-go using a causal self-attention mechanism. The ME 780 study found that DTs achieve a cross-track error of 1.717 m and are prone to oscillation if the training dataset is insufficiently diverse. To address this, the MASc research must utilise a dataset that includes recovery maneuvers, either by injecting noise into a DDPG expert’s actions or by explicitly including high-error-state trajectories.

2.3 Perception and Representation Learning

2.3.1 Convolutional Neural Networks for Driving

CNNs have become the standard front-end for autonomous driving perception, extracting spatial features from camera images and occupancy grids. End-to-end learning approaches train CNNs directly from sensor inputs to control outputs, allowing the network to learn which visual features are most relevant for the control task. In the simulation setting of this thesis, CNNs process bird’s-eye view images rendered from the Pygame environment to extract lane geometry and obstacle information.

2.3.2 Autoencoders for State Compression

Autoencoders compress high-dimensional observations into a compact latent representation, mitigating the curse of dimensionality for RL training. By pre-training an autoencoder on images collected from simulation rollouts, the encoder learns to capture essential geometric features of the road and obstacles. The RL agent then operates on the latent vector z , which reduces policy network parameters, accelerates convergence, and improves generalisation across environments.

2.4 Safety in Autonomous Systems

Safety of the Intended Functionality (SOTIF) defines the safety boundary for autonomous systems operating under nominal sensor and actuator conditions. For articulated vehicles, a key safety concern is the jackknife condition: the RL reward function must penalise large articulation angles to discourage the agent from entering unstable configurations. Control Barrier Functions (CBFs) offer a formal mechanism to overlay hard safety constraints on top of a learned policy, and are identified as a direction for future work beyond the simulation-based experiments of this thesis.

2.5 Benchmarking Against Recent Waterloo Theses

To contextualise the expected level of rigor, Table 2.1 summarises recent MASc theses from the Mechatronics domain at the University of Waterloo.

Table 2.1: Selected recent MASc theses, University of Waterloo Mechatronics Engineering

Author (Year)	Topic	Methodology	Rigor Highlights
Joseph (2025)	Gaze-enabled grasping assistance	ROS2, Kinova Arm, Meta Quest Pro	User study with 30 participants; workload and performance metrics
Thibault (2022)	Humanoid robotic manipulation	EUROBENCH, REEM-C robot	Defined “loco-manipulation”; new manipulability-stability metrics
Hart (2023)	Gait detection via machine learning	Wearable sensors, ternary classification	6 classical + 1 Transformer model; grid-search hyperparameter optimisation
Wei (2021)	Path following for manipulators	Transpose Jacobian control	Free of inverse transformations; validated on 2-DoF and 4-DoF robots
Bickford (2019)	Autonomous bicycle stabilisation	Linear control, Whipple model	Multi-loop architecture; “virtual paths” for high-error recovery

Common themes include strong mathematical modelling foundations and comprehensive validation datasets. For this thesis, a simple “it works” result is insufficient; the

candidate must quantify stability limits, evaluate performance across varied paths, and compare against classical benchmarks.

Chapter 3

Mathematical Modeling and Problem Formulation

3.1 Tractor Dynamics

The tractor longitudinal and lateral dynamics are:

$$\dot{x} = v_x \cos \psi - v_y \sin \psi \quad (3.1)$$

$$\dot{y} = v_x \sin \psi + v_y \cos \psi \quad (3.2)$$

where v_x is the longitudinal velocity, v_y is the lateral velocity, and ψ is the vehicle heading.

3.2 Trailer Articulation and Reverse Instability

The articulated system introduces hitch geometry defined by:

- L_h : distance from tractor rear axle to hitch point
- L_t : distance from hitch point to trailer axle
- γ : articulation angle between tractor and trailer

The hitch angle kinematics during reverse motion are:

$$\dot{\gamma} = \frac{v}{L_t} \sin \gamma - \frac{v}{L_h} \tan \delta \cos \gamma \quad (3.3)$$

Reverse motion introduces instability as γ approaches the jackknife threshold.

3.3 Challenges of Reverse Maneuvering

Reverse motion is characterised by a sign reversal in the kinematic equations, which transforms the stabilising effect of forward velocity into a destabilising force. Successful reverse maneuvering requires anticipating the steering input needed to prevent γ from reaching its mechanical limits. The RL agent developed in this thesis must learn analogous foresight: rewarding actions that keep the trailer aligned with the target path while maintaining γ within a safe operational envelope.

3.4 MDP Formulation

The control problem is cast as a Markov Decision Process (MDP) $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma_d)$ where:

- $\mathcal{S}$: observation space consisting of a vector of state variables (steering angle, articulation angle, cross-track error, heading error) and an 84×84 grayscale BEV image;
- $\mathcal{A}$: continuous action space $[\dot{\delta}, v] \in [-15^\circ/\text{s}, 15^\circ/\text{s}] \times [0, 2 \text{ m/s}]$;
- $\mathcal{T}$: transition dynamics simulated in the custom Pygame/Gymnasium environment;
- $\mathcal{R}$: reward function defined in Chapter 5;
- γ_d : discount factor.

Episodes terminate on collision, jackknife ($|\gamma| > 90^\circ$), or successful path completion.

Chapter 4

Perception and Observation Space Design

4.1 Observation Modalities

The simulator now supports three observation families so that perception can be studied as an ablation rather than a fixed design choice:

- **State-only:** an 8-dimensional vector containing steering angle, hitch angle, tractor cross-track and heading error, trailer cross-track and heading error, and two local curvature terms.
- **State + lidar:** the same 8-dimensional state vector augmented with N normalised range readings, where $N \in \{8, 16, 24\}$ in the current Phase 1 experiments.
- **State + BEV image:** a dictionary observation with the state vector and an 84×84 grayscale bird’s-eye-view image.

This design extends the prior ME 780 work, which relied on compact vector observations only, and allows the thesis to compare hand-engineered geometric sensing against learned visual representations within the same tractor-trailer task.

4.2 Bird’s-Eye View Observation

The BEV image is generated by a configurable camera anchored to the tractor’s rear axle, rendering a top-down 84×84 grayscale image of the vehicle and its surroundings from the Pygame simulation. The anchor point and zoom level are configurable in the simulation settings. The image encodes lane markings, obstacle positions, local path geometry, and obstacle placement, providing spatial context that the vector observation alone cannot capture.

Forward and reverse variants of the BEV environment share the same image format. In reverse manoeuvres, the policy still receives the same image representation, but the vector component is interpreted with trailer-led tracking errors because the trailer becomes the leading unit during backward motion.

4.3 CNN Encoder

A Convolutional Neural Network (CNN) serves as the front-end of the RL agent, processing the BEV image to extract spatial features relevant to lane tracking and obstacle avoidance. The CNN enables the agent to perceive the environment directly from rendered images, without requiring a pre-defined symbolic representation of every object.

The CNN architecture is implemented as a custom feature extractor within the Stable Baselines3 framework. A NatureCNN-style image encoder maps the grayscale BEV image to a 3136-dimensional feature vector, while a separate two-layer multilayer perceptron embeds the vector state to 64 dimensions. The two branches are concatenated before being passed to the TD3 actor and critic networks. This combined observation allows the policy to leverage both precise geometric state information and rich spatial context.

The CNN is trained end-to-end with the TD3 agent, allowing the feature extractor to learn which visual features are most relevant for path maintenance and collision avoidance.

4.4 Autoencoder Latent Space

To improve RL training efficiency, an autoencoder can be used to compress high-dimensional BEV image observations into a compact latent representation. The encoder $E(\cdot)$ maps the image observation s to a latent vector z , and the decoder $D(\cdot)$ reconstructs the original image:

$$z = E(s) \tag{4.1}$$

$$\hat{s} = D(z) \tag{4.2}$$

The autoencoder is pre-trained on images collected from simulation rollouts, allowing the encoder to learn compact representations of the road geometry and obstacle configurations. The current implementation collects BEV images from a mixture of simple pure-pursuit rollouts and any already-trained compatible policies, increasing state coverage during pretraining. After reconstruction training, the encoder weights are transferred into the RL feature extractor and frozen while the policy is trained. This produces an ablation between end-to-end image learning and fixed latent representations learned before RL.

4.5 UNet Encoder Variant

In addition to the CNN autoencoder, the codebase now includes a UNet-style autoencoder for BEV representation learning. The encoder uses repeated double-convolution blocks with max pooling, followed by global average pooling to produce a compact 128-dimensional image descriptor. During pretraining, the full UNet decoder retains skip connections to improve image reconstruction quality; for RL, only the encoder path is kept and concatenated with the 64-dimensional vector embedding.

This second pretrained encoder provides a stronger structural prior than the plain CNN autoencoder. It is intended to test whether preserving multi-scale spatial detail during representation learning improves downstream control performance, particularly in scenes where obstacle shape and lane boundaries must both be interpreted from the BEV image.

4.6 Lidar Observation

The obstacle environments expose a simulated planar lidar observation by appending normalised obstacle distances to the state vector. In forward driving the lidar pose is attached to the tractor, while in reverse driving it is attached to the trailer and rotated by π so that the sensing direction follows the leading unit. This design keeps the sensing model consistent with the physical task: the front of the articulated system changes when the direction of travel reverses.

4.7 Observation Space Summary

The full observation interface used across experiments is summarised in Table 4.1.

Table 4.1: Observation space components

Component	Description	Role
vector	8-dimensional state: $[\delta, \gamma, e_y, e_\psi, e_{y,t}, e_{\psi,t}, \kappa_1, \kappa_2]$	Provides precise articulation and path-tracking state information
lidar	N normalised obstacle ranges, typically $N \in \{8, 16, 24\}$	Encodes local free space around the leading unit without image processing
image	84×84 grayscale BEV image rendered from Pygame simulation	Encodes lane layout, local path shape, obstacle positions, and vehicle geometry

Collision detection is performed using Pygame sprite mask overlap, enabling pixel-precise detection of contact between the vehicle body and obstacles or lane boundaries. Episode termination is triggered on collision, jackknife ($|\gamma| > 90^\circ$), or path completion.

Chapter 5

Reinforcement Learning Framework

5.1 Twin Delayed Deep Deterministic Policy Gradient (TD3)

Twin Delayed Deep Deterministic Policy Gradient (TD3) is an off-policy actor-critic algorithm designed for continuous action spaces. TD3 extends the Deep Deterministic Policy Gradient (DDPG) algorithm from the ME 780 prior work with three key improvements: twin critic networks (to reduce overestimation bias), delayed policy updates (to improve stability), and target policy smoothing (to reduce variance in critic updates).

Like DDPG, the TD3 actor network $\mu(s \mid \theta^\mu)$ maps states to deterministic actions, while the critic networks $Q_1, Q_2(s, a \mid \theta^{Q_1}, \theta^{Q_2})$ evaluate action quality. The target value is computed using the minimum of the two critic estimates:

$$y = r_t + \gamma_d \min_{i=1,2} Q_i(s_{t+1}, \mu(s_{t+1})) + \epsilon \quad (5.1)$$

where $\epsilon \sim \text{clip}(\mathcal{N}(0, \sigma), -c, c)$ is clipped Gaussian noise added to the target action for smoothing. Parameters are updated by minimising the temporal difference error in the critics and maximising the expected return through the actor using the deterministic policy gradient theorem.

TD3 is implemented using the Stable Baselines3 library, which provides a reliable implementation integrated with the Gymnasium environment API.

5.1.1 Reward Function

The proposed reward function is:

$$R = w_1 e^{-|e_y|} + w_2 e^{-|e_\psi|} - w_3 |\gamma| - w_4 C \quad (5.2)$$

where:

- e_y = trailer cross-track error
- e_ψ = heading error
- γ = articulation angle
- C = collision penalty

A multiplicative alternative is also considered:

$$R = e^{-|e_y|} \cdot e^{-|e_\psi|} \quad (5.3)$$

5.1.2 Multiplicative vs. Additive Reward Structures

Prior ME 780 results indicated that a multiplicative reward structure forces the agent to minimise both cross-track and heading errors simultaneously, preventing the policy from exploiting partial credit in the additive form. In this thesis, this logic is extended to include hitch articulation stability: the multiplicative penalty on $|\gamma|$ discourages the agent from trading path accuracy for a large articulation angle.

5.2 Decision Transformer

The Decision Transformer (DT) reformulates RL as a sequence-to-sequence problem. Given a context window of past states $s_{t-k:t}$, actions $a_{t-k:t-1}$, and returns-to-go $\hat{R}_{t-k:t}$, the model predicts the next action using the causal self-attention mechanism of the Transformer architecture:

$$a_t = \pi_\theta(s_{t-k:t}, a_{t-k:t-1}, \hat{R}_{t-k:t}) \quad (5.4)$$

Decision transformers are an offline reinforcement learning approach: rather than training by interacting with the environment, they are trained on recorded trajectories of an agent (such as the TD3 policy) interacting with the environment. This architecture is relevant for the tractor-trailer setting, where early steering errors may only result in a jackknife condition several seconds later, requiring the model to reason over long-term dependencies.

5.2.1 Dataset Requirements

The ME 780 study revealed that DTs are prone to oscillation if the training dataset is too narrow in state diversity. To address this, training data should include recovery manoeuvres obtained by:

- injecting noise into a TD3 expert’s actions during data collection;

- explicitly including trajectories initialised from high-error states (e.g., $|e_y| > 1\text{ m}$ or $|\gamma| > 20^\circ$);
- including step discontinuities in the reference path to force the model to learn re-acquisition behaviour, as demonstrated in the ME 780 work.

5.3 Hyperparameter Optimisation

Both the TD3 and DT agents require careful hyperparameter tuning. The actor-critic learning rates, replay buffer size, target network update rate, and noise scale are selected over simulation rollouts. For the TD3 experiments, the codebase now includes a dedicated Phase 1 training pipeline that trains matched agents across state-only, lidar, and BEV observation variants. BEV policies use custom multi-input feature extractors, while the state-only and lidar policies use standard multilayer perceptron policies.

To reduce wall-clock cost for image-based training, the training script supports vectorised environment execution with subprocess workers. When multiple environments are used, the data collection schedule switches from episode-based to step-based updates so that parallel workers remain saturated instead of waiting for all episodes to terminate simultaneously. This is particularly important for the CNN-based BEV policy, whose throughput is otherwise dominated by environment interaction rather than gradient computation.

5.3.1 Representation Pretraining

Two of the BEV variants use an explicit pretraining stage before TD3 optimisation. The autoencoder-based model freezes a pretrained NatureCNN encoder, while the UNet-based model freezes a pretrained UNet encoder after transfer from the reconstruction network. Both pretraining procedures operate on BEV images collected from policy rollouts and simple geometric controllers. This separates representation learning from control learning and allows a direct comparison against the end-to-end CNN baseline.

5.3.2 Classical Controller Tuning

The thesis now includes a reproducible tuning pipeline for the classical baselines used in later comparison chapters. Forward and reverse pure-pursuit, PID, and MPC controllers are optimised using differential evolution over repeated simulation episodes. The scalar objective penalises trailer cross-track error, jackknife events, collisions, and non-completion. After optimisation, each controller is validated on held-out seeds and a two-dimensional sensitivity sweep is generated over the most influential parameters. This

ensures that any comparison against TD3 is made against tuned baselines rather than hand-picked nominal gains.

Chapter 6

Simulation Environment

6.1 Custom Pygame/Gymnasium Environment

The training and evaluation environment is a custom 2D simulation built on the Farama Foundation Gymnasium library, consistent with the approach taken in the ME 780 prior work. Gymnasium provides a standardised API for implementing environments that integrate directly with reinforcement learning algorithm libraries, including Stable Baselines3.

The simulation is implemented using Pygame for rendering and physics integration. At each timestep, the discrete-time tractor-trailer state-space equations are re-evaluated using the previous state and control inputs, updating the vehicle position, heading, hitch angle, and associated error signals. Pygame renders the scene as a 2D top-down view, from which the BEV image observation is captured.

The simulation environment includes:

1. **Vehicle Model:** A combined dynamic tractor (bicycle model with Pacejka tire dynamics) and kinematic trailer with hitch constraints, using Tesla Model S parameters as in the ME 780 and ECE 682 prior work.
2. **Lane Generation:** Randomly generated lane paths using cubic spline interpolation, producing varying curvature paths for training and evaluation.
3. **Obstacle Placement and Local Replanning:** Randomly placed static obstacles within the lane and a local path planner that perturbs the centreline around those obstacles for avoidance experiments.
4. **Collision Detection:** Pygame sprite mask-based collision detection, providing pixel-precise contact sensing between the vehicle body and obstacles or lane boundaries.

5. **BEV Camera:** A configurable bird’s-eye view camera anchored to the tractor’s rear axle, rendering an 84×84 grayscale image of the vehicle and its surroundings.

6.2 Environment Tasks

The environment implementation is structured around reusable task and observation components. A base lane-following task is combined with different observation heads (state only, lidar, or BEV), while obstacle-aware variants reuse the same task logic through a mixin that adds obstacle generation, local path planning, and obstacle-aware reward evaluation. This avoids maintaining separate forward, reverse, lidar, and BEV implementations with duplicated task logic.

6.2.1 Line Following

The lane-following environment extends the base tractor-trailer environment to present a lane-tracking task. A reference lane is generated at the start of each episode using cubic spline interpolation through randomly placed waypoints. The reward function encourages the agent to keep both the tractor and trailer within the lane boundaries while maintaining progress along the path.

For forward driving, the observation at each timestep may include:

- **vector:** steering angle δ , hitch articulation angle γ , tractor and trailer cross-track and heading errors, and local curvature cues;
- **lidar:** simulated obstacle distances appended to the vector state when lidar sensing is enabled;
- **image:** an 84×84 grayscale BEV image rendered from the tractor rear axle anchor point.

Reverse line-following environments use the same path generation and rendering pipeline, but reverse the longitudinal command and compute reward and tracking terms with the trailer as the leading unit. Reverse BEV and reverse lidar observation classes are implemented explicitly so that perception experiments can be repeated in backward manoeuvres without changing the task definition.

6.2.2 Obstacle Avoidance

The obstacle avoidance environment adds static obstacles within the lane. The agent must navigate the tractor-trailer through the lane while avoiding obstacles, which requires both path-following behaviour and collision avoidance. Instead of following the global lane

centreline directly, the environment constructs a local path around the placed obstacles and evaluates tracking error relative to this locally planned path. Episode termination occurs on collision, jackknife ($|\gamma| > 90^\circ$), or successful path completion.

Forward and reverse obstacle environments are both available. In addition to the original lidar-based formulation, the codebase now includes BEV obstacle-avoidance variants for both travel directions, enabling direct comparison between range-based and image-based obstacle perception within the same local-planning task.

6.3 Vehicle Parameters

The simulation uses the same vehicle parameters as the ME 780 and ECE 682 prior work, based on Tesla Model S specifications:

Table 6.1: Vehicle parameters used in simulation

Parameter	Value	Parameter	Value
m	1500 kg	l_f	1.2 m
I_z	3000 kg m ²	l_r	1.6 m
C_f	80000 N/rad	C_d	0.208
C_r	80000 N/rad	A	2.4 m ²
Δt	0.1 s	ρ	1.225 kg/m ³

6.4 Action Space

The agent controls the vehicle through a continuous two-dimensional action space:

$$\mathbf{a} = [\dot{\delta}, v] \in [-15^\circ/\text{s}, 15^\circ/\text{s}] \times [0, 2 \text{ m/s}] \quad (6.1)$$

where $\dot{\delta}$ is the steering rate (rate of change of steering angle) and v is the commanded speed. This action parameterisation is consistent with the observation from the ECE 682 work that the steering angle cannot be set instantaneously on a physical vehicle; controlling the rate of change is more physically realistic.

For reverse tasks, the environment constrains the longitudinal command to negative speed while preserving the same steering-rate interface. This keeps the control dimensionality fixed across forward and reverse tasks and allows the same TD3 implementation to be reused.

For thesis evaluation, however, this full action space is not used identically in every study. In the no-obstacle line-following experiments, the longitudinal speed should be treated as a fixed operating condition so that all methods are compared on the same

path-tracking problem. In the obstacle-avoidance experiments, speed modulation remains part of the task because the agent must trade progress against collision risk and path geometry.

6.5 Testing Protocols

The experimental workflow is now scriptable end-to-end. Scenario generation, Phase 1 observation-modality training, trained-policy evaluation, controller tuning, and multi-controller benchmarking are each exposed through standalone scripts. The benchmarking pipeline runs all controllers on the same pre-generated scenario set and logs per-step metrics including tractor and trailer cross-track error, hitch angle, completion, collision, jackknife events, obstacle clearance, and inference latency.

The thesis should treat the experiments as two related but distinct problem classes:

1. **Path-tracking control:** forward and reverse lane following without obstacles. In this setting all methods are solving the same task at a common fixed speed, so TD3 can be compared directly against pure-pursuit, PID, and MPC as lateral/path-tracking controllers.
2. **Integrated navigation:** obstacle-avoidance tasks in which the policy must implicitly combine local planning and control. In this setting a direct comparison against controller-only baselines is not fair unless those baselines are also given a local planner.

This distinction is important for experimental validity. A controller that only tracks a supplied path should not be evaluated against an RL policy that must both choose a collision-free local path and execute it, unless the classical baseline is paired with an equivalent planner.

6.5.1 Lane Following Performance

Table 6.2: Lane following experiment definitions

Experiment	Objective	Success Metric
Path Fidelity (Tractor)	Maintain low CTE for tractor on varying curvature paths	Mean Absolute CTE
Path Fidelity (Trailer)	Maintain low CTE for trailer, ensuring full vehicle stays in lane	Mean Absolute CTE
Hitch Stability	Keep articulation angle within safe bounds	Max $ \gamma $ per episode

6.5.2 Obstacle Avoidance

The obstacle avoidance task tests the agent’s ability to navigate through a lane containing static obstacles. Success is measured by collision rate and the agent’s ability to complete the lane while maintaining hitch stability.

For thesis evaluation, obstacle experiments should be framed as a secondary study rather than merged into the primary controller benchmark. Two defensible comparison strategies are available:

- **Planner-controlled comparison:** provide the classical baselines with the same obstacle-aware local path and evaluate only the downstream tracking problem; or
- **End-to-end navigation comparison:** compare the RL obstacle policy against complete planner+controller stacks, not against controllers in isolation.

If implementation scope is limited, the recommended plan is to keep obstacle avoidance as an RL feasibility extension and reserve the main quantitative comparison for the no-obstacle forward and reverse tracking tasks. Under this plan, no-obstacle experiments use fixed speed for both learned and classical methods, whereas obstacle experiments allow the RL agent to command speed as part of the end-to-end navigation problem.

6.5.3 Comparison with Prior Work

Results are compared against:

- the DDPG model from ME 780 (cross-track error 0.505 m on single-vehicle path following);

- a PID baseline from ME 780 (cross-track error 0.784 m);
- tuned pure-pursuit and PID tractor-trailer baselines implemented in this work;
- an LQR-based baseline from ECE 682;
- an MPC controller implemented using OSQP and tuned on held-out simulation episodes.

Chapter 7

Results and Discussion

7.1 Path Following Performance

The results pipeline has been extended beyond single-policy rollouts to support controlled comparison across observation modalities and controller families. In the current implementation, forward lane-following serves as the Phase 1 benchmark task, with matched TD3 agents trained on state-only, lidar, end-to-end CNN BEV, pretrained autoencoder BEV, and pretrained UNet BEV observations. The evaluation script aggregates tractor and trailer cross-track error, hitch-angle statistics, completion rate, and inference latency across repeated episodes.

This structure is important for the thesis because it separates two questions that were previously entangled: whether the articulated control task benefits from richer sensing, and whether any benefit comes from the sensing modality itself or from the representation-learning pipeline used to process it.

The recommended experiment order is therefore:

1. select the best observation space on forward and reverse lane-following without obstacles;
2. hold that observation fixed and compare candidate reward formulations on the same no-obstacle tasks;
3. benchmark the best TD3 configuration against tuned classical controllers on matched no-obstacle scenarios.

This sequence keeps each ablation focused on one design variable at a time and ensures that the final controller benchmark is performed on the cleanest apples-to-apples task definition. In particular, the no-obstacle benchmark should be run at fixed speed for every method, so that differences in performance are attributed to path tracking and articulation stabilisation rather than to longitudinal speed selection.

Table 7.1 converts that sequence into an executable run matrix. The training launcher supports all observation and reward combinations listed there through the `scenario`, `obs`, and `reward` arguments, while the benchmark scripts are used only after model selection.

Table 7.1: Planned run matrix for the experimental campaign

Study	Scenario	Observation sweep		Reward sweep	Runs	Purpose
Phase 1A	forward	state, BEV	lidar	dense	3	Select best forward observation space
Phase 1B	reverse	state, BEV	lidar	dense	3	Select best reverse observation space
Phase 2A	forward	best Phase 1A	from	dense, tractor_focus, multiplicative	3	Select best forward reward
Phase 2B	reverse	best Phase 1B	from	dense, no_hitch, multiplicative	3	Select best reverse reward
Phase 3	forward, reverse	best RL configuration per direction	con-	benchmark only	2	Compare TD3 against tuned pure pursuit, PID, and MPC
Phase 4	forward_obs, reverse_obs	lidar or BEV		best reward from matching direction	4	Secondary obstacle-navigation study

7.2 Reverse Maneuvering Success

Reverse evaluation now includes dedicated reverse line-following and reverse obstacle-avoidance environments, along with reverse pure-pursuit, reverse PID, and reverse MPC baselines. All reverse controllers are formulated around hitch-angle stabilisation and trailer-referenced tracking errors, reflecting the fact that the trailer leads the manoeuvre during backing. This provides a consistent basis for comparing learned and analytical controllers on the inherently unstable reverse articulated-driving problem.

7.3 Perception Robustness

The perception study now has a concrete ablation structure: state variables only, state plus lidar, end-to-end BEV learning, and frozen pretrained BEV encoders. Because all variants share the same TD3 backbone and task definition, differences in tracking performance can be attributed more directly to the observation and representation choices than to changes in the control architecture.

In practical terms, the most meaningful observation-space study is the one run on the no-obstacle environments. Obstacle environments confound the comparison because different observation spaces change not only path-tracking quality but also how much implicit local-planning information is available to the policy. They also change how much benefit a method can obtain from modulating speed, which is undesirable in the core controller comparison.

7.4 Benchmark Comparison

Table 7.2: Controller performance comparison

Model	Avg. CTE (m)	Jackknife Events/100	Latency (ms)
PID	0.784	High	< 5
NMPC	0.350	Low	50–100
DDPG	0.505	Medium	10–20
Decision Transformer	1.717	High	30–50

Benchmarking is performed on pre-generated scenario sets so that TD3 and the classical baselines face identical lane geometry and obstacle placement. Classical controllers are tuned before comparison using differential evolution and then validated on held-out seeds. Each benchmark episode logs both control quality and computational cost, allowing the thesis to compare not only tracking error but also completion rate, jackknife frequency, collision rate, obstacle clearance, and controller latency.

The results must be understood in terms of multi-objective performance: in rigid-body systems, success is synonymous with minimising cross-track error. In articulated systems, hitch stability takes precedence over exact path following. An agent that maintains $|\gamma| < 5^\circ$ throughout a reverse task is more successful than one that achieves lower CTE but triggers jackknife events. For that reason, the benchmark framework reports trailer tracking, articulation stability, and completion together rather than reducing performance to a single scalar.

For obstacle scenarios, the benchmark question changes. An RL obstacle policy is effectively learning a joint local-planning and control policy, including speed modulation,

whereas pure-pursuit, PID, and MPC are primarily path-tracking controllers. The thesis should therefore avoid presenting obstacle-task results as if they were directly comparable to controller-only baselines unless a matched planning layer and longitudinal policy are added to those baselines. Obstacle results are best reported either as a separate end-to-end navigation study or as a comparison against planner+controller classical stacks.

Table 7.3: Recommended experiment matrix for the thesis

Experiment	Task	Observation		Success Metric	
Observation ablation (forward)	Path following	state, BEV	lidar,	Fixed-speed mean abs. CTE, max $ \gamma $, completion	trailer
Observation ablation (reverse)	Reverse tracking	state, BEV	lidar,	Fixed-speed mean abs. CTE, max $ \gamma $, jackknife rate	trailer
Reward ablation (forward)	Path following	best observation only	observa-	Fixed-speed mean return, trailer CTE, stability metrics	re-
Reward ablation (reverse)	Reverse tracking	best observation only	observa-	Fixed-speed completion, jackknife rate, trailer CTE	comple-
Controller benchmark (forward)	Path following	best RL obs. vs. classical	vs.	Fixed-speed TD3 vs. pure pursuit vs. PID vs. MPC	TD3 vs.
Controller benchmark (reverse)	Reverse tracking	best RL obs. vs. classical	vs.	Fixed-speed TD3 vs. reverse pure pursuit vs. reverse PID vs. reverse MPC	TD3 vs.
Obstacle RL extension	End-to-end navigation	lidar or BEV		Variable-speed collision rate, completion, min clearance	collision
Obstacle classical comparison	Planner + controller	matched path + controller	local + con-	Only if classical baselines receive equivalent planning information	troller

Chapter 8

Conclusion

8.1 Summary of Findings

This thesis presented a reinforcement learning framework for articulated autonomous vehicle control, building on the foundational work from ME 780 (DDPG and Decision Transformer for single-vehicle path following) and ECE 682 (LQR control of the vehicle bicycle model with tractor-trailer kinematics). The key contributions demonstrated are:

- a combined dynamic tractor and kinematic trailer model implemented in a custom Gymnasium simulation environment, extending the single-vehicle bicycle model from prior work;
- a vision-based observation space incorporating BEV images rendered from the Pygame simulation, enabling spatial perception of lane boundaries and obstacles;
- a TD3 reinforcement learning agent trained on lane-following and obstacle avoidance tasks with a multiplicative reward function penalising cross-track error, heading error, and hitch articulation angle;
- comparison of TD3 performance against PID and MPC classical control baselines and the prior ME 780 DDPG results.

The integration of visual perception via BEV imagery extends the state-only approach of the ME 780 work. By processing rendered BEV images through a CNN feature extractor, the agent gains spatial context about lane geometry and obstacle positions that vector state observations alone cannot provide.

8.2 Limitations

The study is subject to the following constraints:

- Experiments are conducted entirely within the custom 2D Pygame/Gymnasium simulation environment; performance in higher-fidelity simulators or on physical hardware has not yet been characterised.
- The 2D simulation approximates the vehicle dynamics but does not capture all effects present in a real tractor-trailer system, such as load transfer, tire saturation at high slip angles, or hitch joint compliance.
- The current framework assumes a fixed payload; hitch dynamics change significantly with varying trailer mass.
- Obstacle avoidance is limited to static obstacles; dynamic obstacle handling is not addressed.

8.3 Future Work

Several directions remain open for future investigation:

- **Higher-fidelity simulation:** Validating the trained policies in a physics-based 3D simulator (e.g., Gazebo) with a detailed URDF model of the tractor-trailer system, including realistic tire and hitch dynamics.
- **Hardware deployment:** Transferring trained policies to a physical robotic platform, requiring sim-to-real transfer techniques such as domain randomisation and system identification.
- **Planner-controller baselines for obstacle tasks:** Comparing obstacle-avoidance RL policies against complete classical navigation stacks, so that end-to-end navigation is evaluated on a fair task definition rather than against controller-only baselines.
- **Task-specific action constraints:** Separating fixed-speed path-tracking experiments from variable-speed obstacle-navigation experiments in the implementation, so that the thesis benchmark and the codebase enforce the same problem definition.
- **Decision Transformer for tractor-trailer:** Applying the DT approach from ME 780 to the articulated vehicle setting, with training data generated from the TD3 expert policy and augmented with high-error recovery trajectories.
- **Multi-trailer systems:** Extending the kinematic model and RL policy to double or B-train configurations.
- **Online adaptation:** Updating the policy in real time to compensate for varying payloads and road conditions.

- **Control Barrier Function overlays:** Implementing CBFs as a hard safety layer that overrides the RL agent when a collision or jackknife is imminent.
- **Dynamic obstacles:** Extending the obstacle avoidance environment to include moving obstacles and integrating predictive collision avoidance.

Appendix A

Simulation Software Architecture

Appendix B

Full Hyperparameter Tables

Table B.1: TD3 Hyperparameters

Hyperparameter	Value
Learning rate	
Batch size	
Replay buffer size	
Policy noise	
Target update rate	
Timesteps	200,000

Appendix C

Additional Plots

Appendix D

Reward Function Derivation

Bibliography